Trabajo Práctico N1

NÓMADA VIAJES

Aplicaciones web 1

Grupo: Agorio Lautaro, Giovannetti Juan Ignacio y Rossi Tiziana.

Índice

Índice **2**
1.1. Descubrimiento y ficha técnica del producto **3**
1.1.1. Datos del cliente 3
1.1.2. Objetivos del proyecto web 3
1.1.3. Descripción del producto web y modelo de negocio mínimo 3
1.1.4. Proto-persona e historias de usuario 4
1.1.4.1. Proto-persona 4
1.1.4.2. Historias de usuario 4
1.1.4.3. Flujo de usuario (User Flow) 4
1.1.5. Requerimientos funcionales frontend 5
1.2. Arquitectura de la información **5**
1.2.1. Sitemap (Mapa del sitio) 5
1.2.2. Estrategia de navegación (MPA / SPA) 5
1.3. Diseño de interfaz (UX / UI) **7**
1.3.1. Bocetos a mano 7
1.3.2. Wireframes 9
1.3.3. Mockup 14
1.4. Infraestructura web y dominio **14**
1.4.1. Selección de dominio 14
1.5. Correcciones **15**
1.5.1. Modificaciones 15
1.5.2. Comentarios sobre el desarrollo HTML 16
1.2.3. Comentarios sobre el desarrollo CSS 16
1.6. Bibliografía **18**

1.1. Descubrimiento y ficha técnica del producto

1.1.1. Datos del cliente

Nombre / Marca: Nómada Viajes “Tu próximo destino, a un clic de distancia”

Tipo de cliente: Ficticio.

Rubro: Agencia de viajes online, orientada a la venta de paquetes turísticos combinados (vuelos, alojamientos y excursiones) para viajeros individuales, parejas y grupos familiares.

1.1.2. Objetivos del proyecto web

Nómada Viajes busca digitalizar por completo el proceso de venta de paquetes turísticos, permitiendo que el usuario busque, compare y reserve sus próximas vacaciones sin necesidad de asistir presencialmente a la agencia ni depender de la atención telefónica. La plataforma centraliza la oferta de destinos y paquetes en un catálogo único, actualizado y filtrable por destino, fecha y presupuesto, y busca reducir el tiempo de gestión de consultas y reservas mediante formularios de contacto y confirmaciones automáticas. Otro objetivo central es ampliar el alcance geográfico de la agencia, captando clientes de todo el país a través de la web sin depender exclusivamente de la oficina física, y generar confianza en el proceso de compra online mostrando de forma clara el detalle de cada paquete y confirmando cada paso de la reserva.

1.1.3. Descripción del producto web y modelo de negocio mínimo

Nómada Viajes será una plataforma e-commerce B2C que permite a los usuarios explorar un catálogo de paquetes turísticos, ver el detalle completo de cada uno (itinerario, qué incluye, precio y disponibilidad), armar su selección en un carrito de compras, completar un formulario con los datos del viajero y confirmar la reserva y el pago de forma online, recibiendo la confirmación por el mismo canal.

Modelo de negocio:

- Comisión por la venta de paquetes de operadores mayoristas y aerolíneas.
- Venta directa de paquetes propios armados por la agencia (destino + alojamiento + excursiones).
- **Venta de servicios adicionales:** seguros de viaje, traslados aeropuerto–hotel y excursiones opcionales no incluidas en el paquete base.
- Señas o pagos en cuotas para reservas anticipadas (reserva con seña online y saldo posterior).

1.1.4. Proto-persona e historias de usuario

1.1.4.1. Proto-persona

Nombre: Julieta Sanchez

Edad: 31 años

Ocupación: Diseñadora gráfica freelance

Dispositivo principal: Smartphone (Android)

Otros dispositivos: notebook personal, usada para trámites puntuales como pagos y formularios

Comportamiento y hábitos:

- Organiza sus vacaciones con 2 o 3 meses de anticipación.
- Compara precios y fechas en varias webs antes de decidir.
- Sigue cuentas de viajes en Instagram y guarda destinos que le interesan.
- Prefiere pagar en cuotas con tarjeta de crédito.

Objetivos y necesidades:

- Encontrar paquetes que se ajusten a su presupuesto sin tener que escribir a varias agencias.
- Viajar con su pareja una o dos veces al año.
- Tener la reserva confirmada por escrito, sin depender de una respuesta por WhatsApp.

Frustraciones principales:

- **Falta de comparación ágil:** le cuesta comparar precios entre distintas fechas y destinos porque hoy tiene que escribir por WhatsApp a la agencia y esperar respuesta.
- **Incertidumbre tras el pago:** le preocupa no tener certeza de que la reserva quedó realmente confirmada después de pagar una seña.

1.1.4.2. Historias de usuario

- **Historia 1:** Como usuaria interesada en viajar, quiero buscar y filtrar paquetes por destino, fecha y precio, para encontrar rápidamente una opción que se ajuste a mi presupuesto y al tiempo que tengo disponible.

- **Historia 2:** Como usuaria que ya reservó un paquete, quiero recibir una confirmación inmediata de mi reserva y de mi pago, para no quedarme con la incertidumbre de si mi viaje quedó realmente confirmado.

1.1.4.3. Flujo de usuario (User Flow)

Flujo principal, desde el ingreso al sitio hasta completar la reserva:

Inicio ⟶ Catálogo de paquetes ⟶ Filtrar (destino / fecha / precio) ⟶ Detalle del paquete ⟶ Agregar al carrito ⟶ Completar datos del viajero y pagar seña ⟶ Confirmación de reserva

1.1.5. Requerimientos funcionales frontend

- Menú de navegación responsive colapsable (menú hamburguesa) en dispositivos móviles.
- Buscador y filtros en tiempo real por destino, rango de fechas, rango de precio y tipo de viaje (playa, aventura, ciudad).
- Carrito de compras dinámico que permita agregar y quitar paquetes, y recalcule el total sin recargar la página.
- Galería o carrusel de imágenes en el detalle de cada paquete, y carrusel de destinos destacados y ofertas vigentes en la página de Inicio.
- Formulario de reserva/consulta con validación de campos obligatorios mediante JavaScript (nombre, DNI, email, teléfono).
- Sistema de notificaciones: confirmación de reserva, aviso de pago acreditado y alertas de ofertas para destinos guardados como favoritos.

1.2. Arquitectura de la información

1.2.1. Sitemap (Mapa del sitio)

Inicio (Home)

- Presentación de la agencia, buscador rápido, destinos destacados, ofertas vigentes, CTA "Ver paquetes"

Catálogo de paquetes

- Grilla de paquetes con filtros (destino, fecha, precio, tipo de viaje)
- Detalle de paquete (itinerario, qué incluye, precio, fotos, botón "Agregar al carrito")
- Carrito de compras
- Checkout / confirmación de reserva y pago

Nosotros

- Historia de la agencia, equipo y misión

Contacto / Consultas

- Formulario de consulta, datos de contacto, mapa de la oficina

Mi Cuenta (área del cliente)

- Perfil, mis reservas, historial de pagos, notificaciones

Login / Registro

- Inicio de sesión, alta de cuenta nueva, recuperar contraseña

1.2.2. Estrategia de navegación (MPA / SPA)

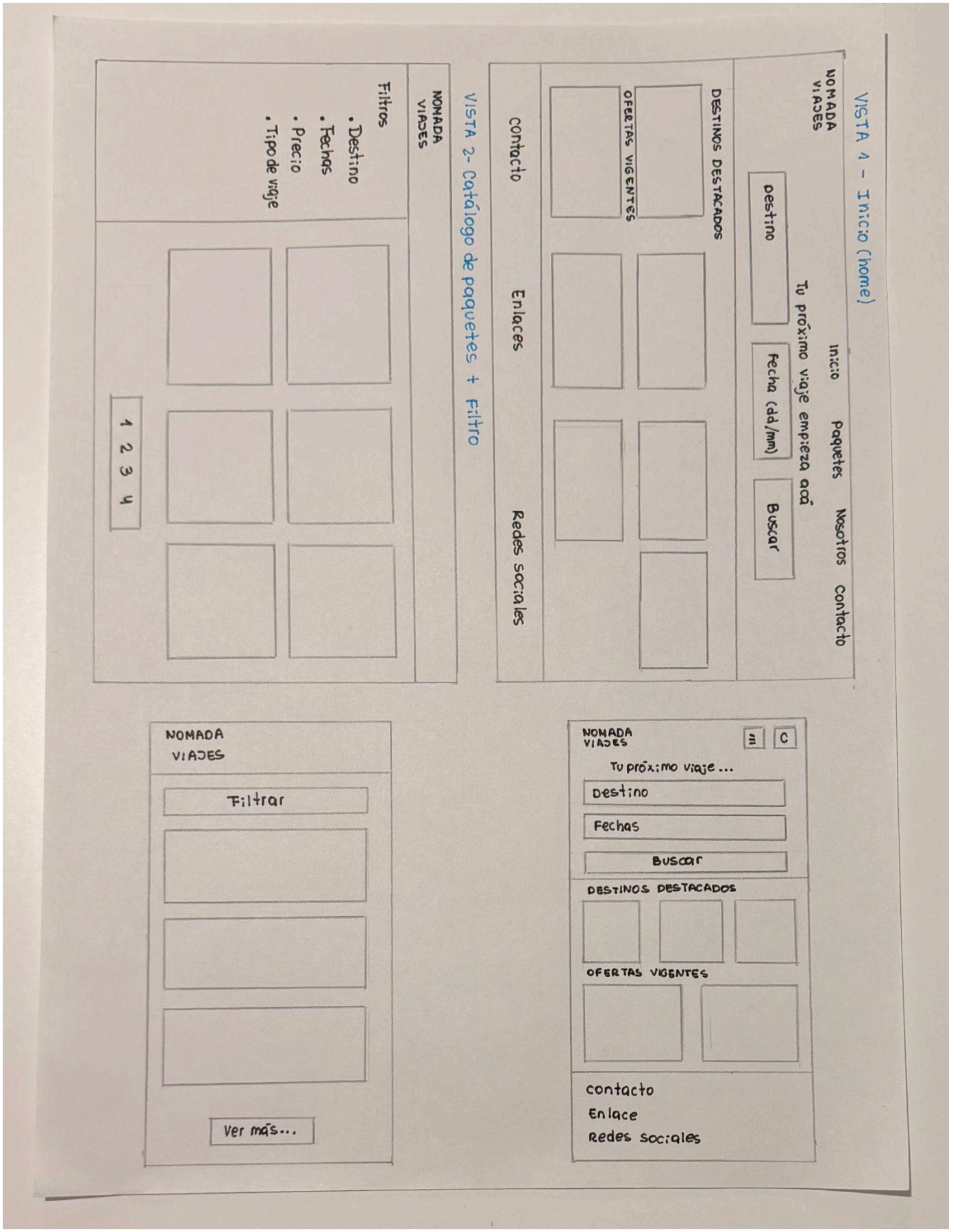
Enfoque	Página / módulo	Justificación
MPA (Multi-Page)	Inicio, Nosotros, Contacto, Login / Registro	Son páginas de bajo cambio y alto valor de indexación (SEO), conviene que Google las rastree por separado para que un usuario nuevo llegue directo desde el buscador. Se benefician de una carga inicial simple e independiente.

SPA (Single-Page)	Catálogo de paquetes (con filtros), Carrito, Checkout, Mi Cuenta	Son módulos privados donde importa la fluidez, filtrar y actualizar el catálogo, modificar el carrito o navegar entre pestañas de Mi Cuenta sin recargar la página completa mejora la experiencia y evita perder el contexto de la compra (por ejemplo, durante el pago).

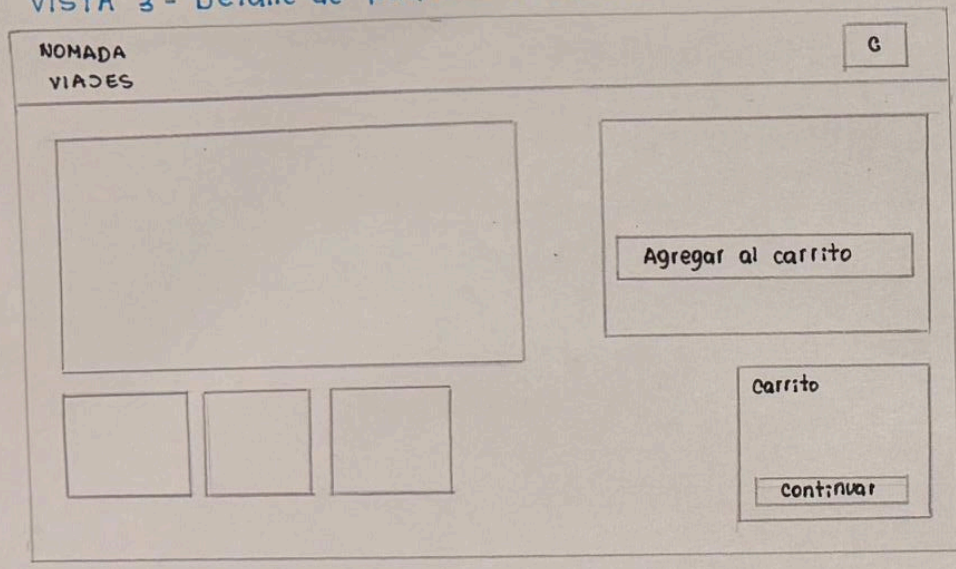
En este proyecto se utilizan ambos enfoques de forma combinada ya que, las páginas informativas y de bajo cambio se resuelven como MPA para priorizar SEO y velocidad de primera carga, mientras que el catálogo, el carrito y el checkout se comportan como módulos SPA para priorizar la velocidad de respuesta ante cada acción del usuario (filtrar, agregar al carrito, pagar).

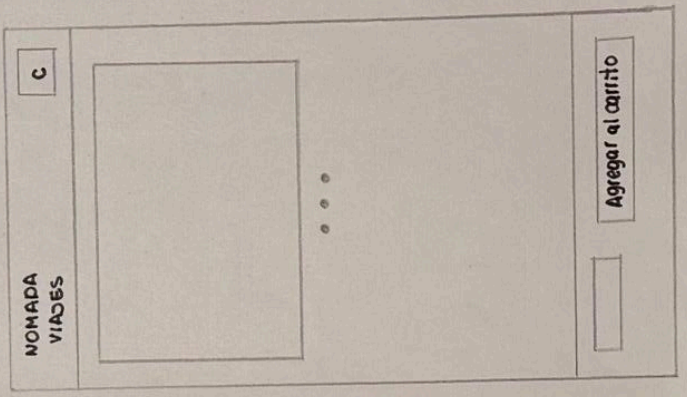
1.3. Diseño de interfaz (UX / UI)

1.3.1. Bocetos a mano

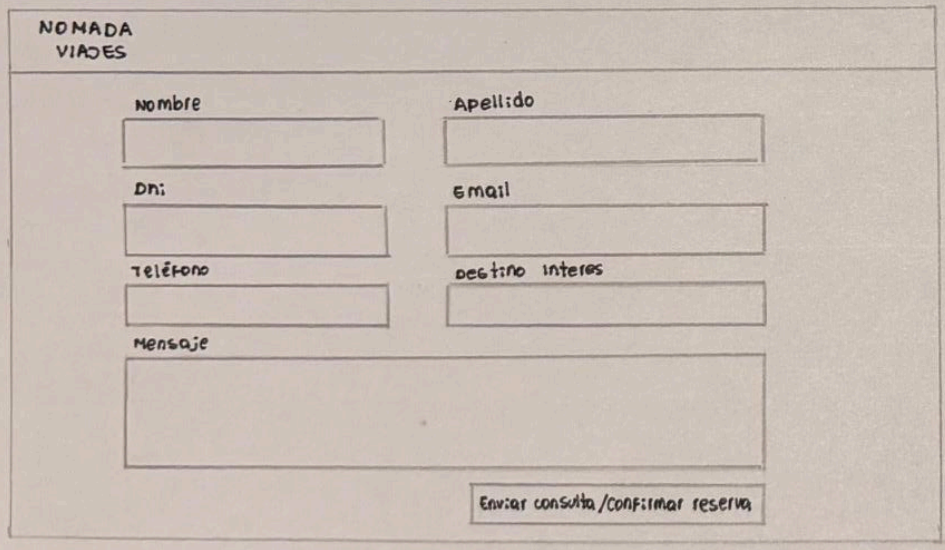


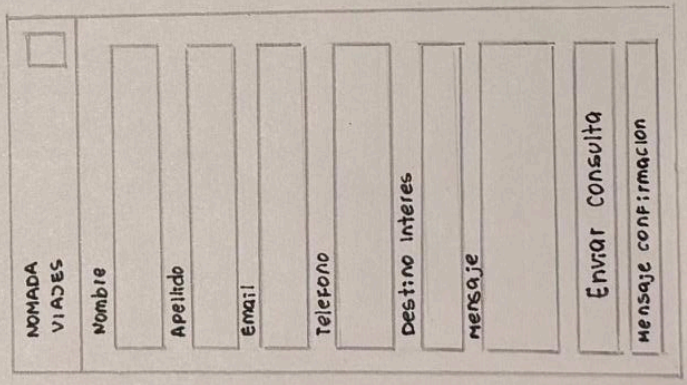
VISTA 3 - Detalle de paquete + carrito





VISTA 4- Formulario de contacto



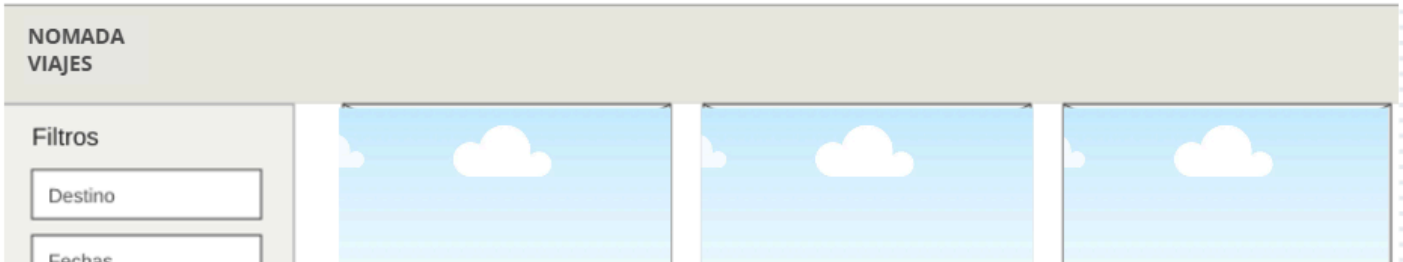


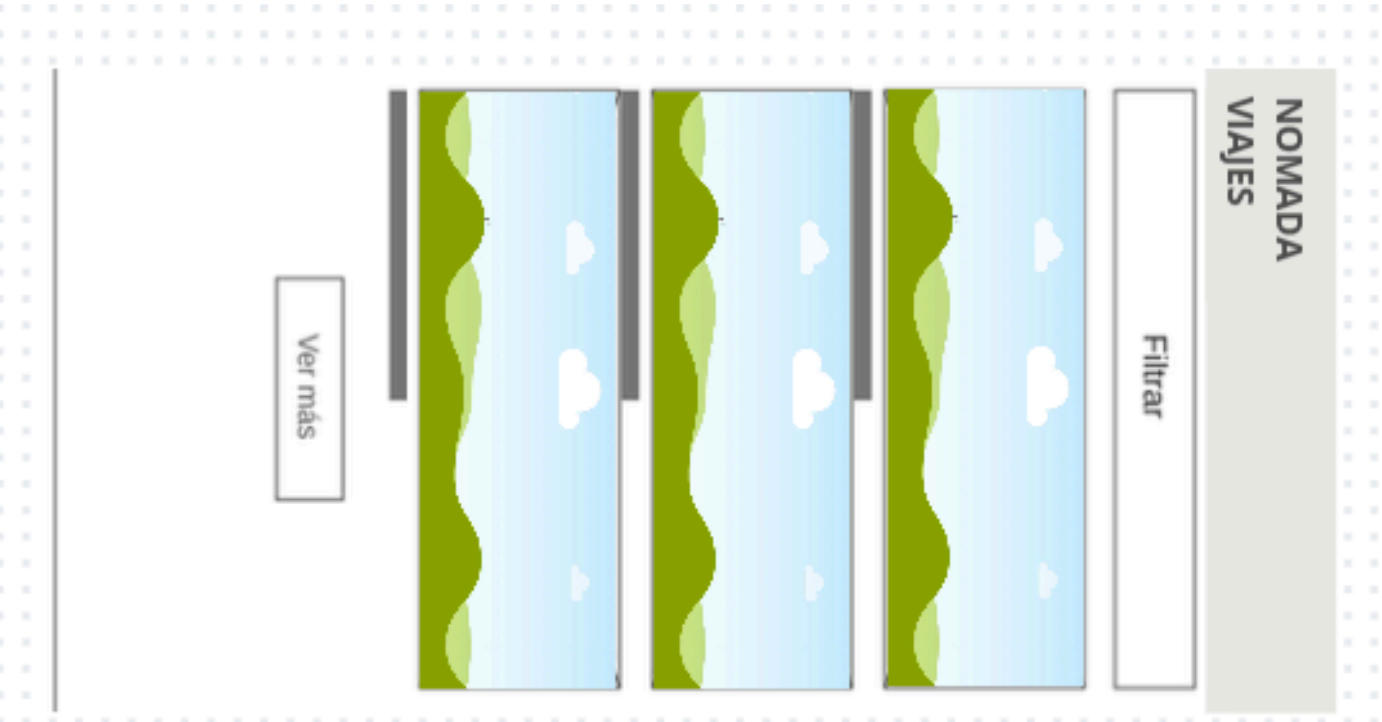
Medidas versión mobile: 390 px - 844 px.

Vista 1 — Inicio (Home)



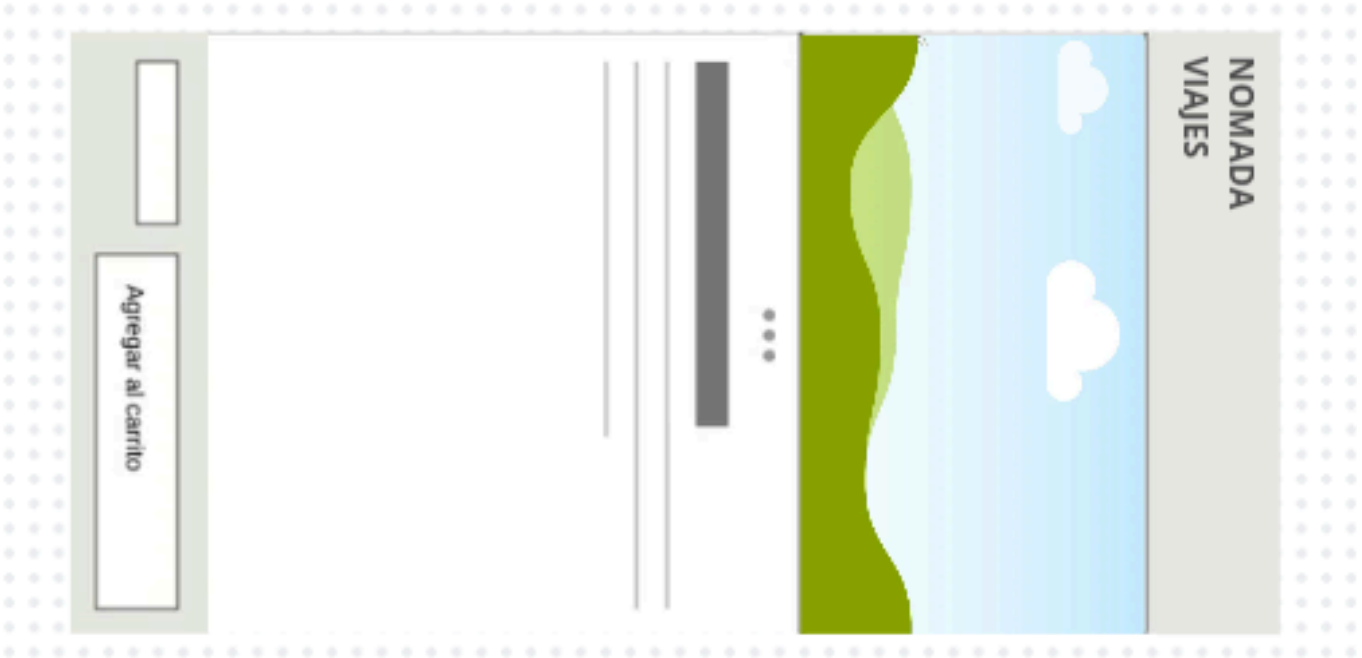






Vista 3 — Detalle de paquete + Carrito





Vista 4 — Formulario de Contacto



Vista 5 — Notificación

NOMADA
VIAJES



¡Reserva confirmada!

N° de reserva: NV-2026-004821

Bariloche al completo — 2 personas

Seña abonada: $126.000

Volver al inicio

https://www.figma.com/design/t0DqTrJHDDCoe4GgJ8bZbl/mockup?node-id=0-1&t=ilt6tg4tSQqo6ay6-1

1.4. Infraestructura web y dominio

1.4.1. Selección de dominio

- **Dominio propuesto:** nomadaviajes.com.ar

- **Registro / Plataforma:** primero verificamos la disponibilidad del dominio en https://nic.ar/ y luego en https://donweb.com/es-ar/dominio-com-ar vemos precio de licencia.



- **Justificación:** se eligió el TLD .com.ar porque refuerza la identidad local de la agencia dentro de Argentina, genera mayor confianza en el usuario a la hora de pagar online (al asociarse a una entidad registrada en el país) y mejora el posicionamiento en buscadores para búsquedas locales del tipo "agencia de viajes y destinos o ciudades".

1.5. Correcciones

Observación del profesor: La observación es que deberían estar los bocetos, wireframes y mockups en un solo documento. Ver también que, en lugar de optimizar el espacio ubicando varios diseños en una página, se ubique un diseño por página para que se vea correctamente.

Respecto al segundo trabajo práctico, se identificó que el formulario de reserva y el catálogo de paquetes empleaban el método “GET”, cuando correspondía implementar el método “POST” para el envío y procesamiento adecuado de los datos ingresados. Asimismo, en el archivo detalle.html se detectó un elemento <ul> que no cumplía con la función estructural de una lista.

1.5.1. Modificaciones

En base a las correcciones del profesor, se añadieron los bocetos y wireframes en este mismo documento y además se ubicaron en diferentes paginas para tener una mejor visualización.

Además, en el punto 1.3.2 de Wireframes se incorporó la Vista 5 Notificación, correspondiente a la pantalla de confirmación de reserva, en sus versiones desktop y mobile. Esta vista no había sido incluida en la entrega anterior, pero la consigna general del trabajo práctico exige que la interfaz cuente como mínimo con tres páginas más las pantallas de notificaciones, por lo que se agregó este wireframe para cubrir por completo ese requisito y mostrar cómo el usuario recibe la confirmación de su reserva y del pago de la seña al finalizar el flujo de compra.

Las imágenes con extensión jpg se modificaron a webp, se optó por el formato WebP debido a que ofrece un menor peso de archivo, mejor compresión y una mayor velocidad de carga sin pérdida de calidad.

Sobre las observaciones del desarrollo de html se modificó lo siguiente:

```
<form action="confirmacion.html" method="post">
```

```
    <ul>
                <li>
                    Precio final: $420.000 por persona
                </li>
                <button type="button">
                    Agregar al carrito
                </button>
            </ul>
```

1.5.2. Modificaciones respecto a TP3 (CSS)

Formulario de filtros del catálogo

Se modificó la estructura del formulario de filtros de la página de catálogo (catalogo.html) para que tuviera el mismo formato que el formulario de contacto. Cada etiqueta se agrupó con su campo dentro de un contenedor, y las opciones de tipo de viaje se reunieron en un fieldset con su legend. En la hoja de estilos (catalogo.css) se le dio al formulario la misma tarjeta blanca centrada que usa el formulario de contacto, y se organizaron los campos con CSS Grid: en celular se muestran en una sola columna y en pantallas más grandes en dos, de modo que las fechas y los precios quedan uno al lado del otro. Se agregaron además los

atributos id al formulario y al contenedor de los paquetes, para poder utilizarlos más adelante con JavaScript.

Menú hamburguesa para mobile

Se agregó un menú hamburguesa para la versión mobile, que se despliega y se cierra sin utilizar JavaScript. En el encabezado de cada una de las 5 páginas se incorporó un checkbox oculto junto con dos etiquetas (label) asociadas a él, una para abrir el menú y otra para cerrarlo. En la hoja de estilos global (index-global.css), el estado marcado del checkbox permite mostrar u ocultar el menú de navegación y alternar entre el botón de apertura, dibujado con tres barras, y el de cierre, dibujado como una cruz. En pantallas pequeñas el menú se despliega debajo del encabezado y en pantallas de 768px o más se oculta el botón y los enlaces se muestran en una fila, como en la versión de escritorio.

1.5.3. Comentarios sobre el desarrollo HTML

Se llevó a cabo la codificación de la estructura HTML básica de las 5 páginas que conforman el proyecto: Inicio (index.html), Catálogo de paquetes (catalogo.html), Detalle de paquete + Carrito (detalle.html), Formulario de contacto/reserva (formulario.html) y Confirmación de reserva (confirmacion.html). Todas se desarrollaron sin el uso de CSS ni JavaScript, respetando el etiquetado semántico definido en la propuesta: un header con el logo enlazado al inicio y un nav con los accesos a las 5 pantallas del proyecto (Inicio, Paquetes, Detalle y Carrito, Contacto y Confirmación), un main con el contenido específico de cada vista, y un footer común con los datos de contacto, redes sociales y enlaces útiles, repetido de forma idéntica en todas las páginas.

En index.html se codificó el encabezado principal (h1) con la propuesta de valor de la agencia, y las secciones "Destinos destacados" y "Ofertas vigentes" como listas (ul/li), cada ítem con su imagen, nombre del destino y precio.

En catalogo.html se armó el panel de filtros como un formulario (form) con fieldset y legend para agrupar los campos de Destino (select), Fechas (dos input tipo date), Precio (dos input tipo number) y Tipo de viaje (checkboxes agrupados en un fieldset anidado), y debajo la grilla de resultados con tres tarjetas de paquete, cada una con su imagen, cantidad de noches, precio y un enlace "Ver paquete" hacia la vista de detalle.

En detalle.html se desarrolló la vista completa de un paquete (Bariloche al completo), con galería de imágenes (una principal y tres miniaturas), el itinerario día por día como lista ordenada (ol), las listas de "Qué incluye" y "Qué no incluye" (ul), el precio final y un botón "Agregar al carrito". Debajo se codificó el panel del carrito de compras, con el resumen del ítem agregado, el subtotal y un enlace "Continuar" que lleva al formulario de reserva.

En formulario.html se codificó el formulario de reserva dentro de un fieldset, con los 7 campos solicitados (nombre y apellido, DNI, email, teléfono, destino de interés, fecha estimada de viaje y mensaje), cada uno con el tipo de input HTML correspondiente (text, email, tel, date, textarea), y un botón de envío "Confirmar reserva".

En confirmacion.html se codificó la pantalla de notificación final, con un ícono de confirmación, el título "¡Reserva confirmada!", el número de reserva, el resumen del pago

(seña abonada y paquete correspondiente) y un enlace "Volver al inicio" que redirige a index.html.

Como el proyecto no incorpora JavaScript en esta etapa, tanto el botón "Agregar al carrito" como el envío del formulario no ejecutan ninguna acción real de procesamiento de datos: representan únicamente la estructura e interacción prevista, que se implementará en las próximas entregas del trabajo práctico (SP4 y SP5).

1.2.4. Comentarios sobre el desarrollo CSS

Se llevó a cabo la codificación de los estilos de las 5 páginas del proyecto, repartidos en cinco hojas: index-global.css (estilos globales más los de Inicio, vinculada en todas las páginas y siempre primera en el <head>), catalogo.css, detalle.css, formulario.css y confirmacion.css. Cada hoja específica se vincula después de la global y contiene solo los estilos de su vista. Todas siguen el mismo orden: selector universal, selectores de tipo, selectores de clase agrupados por zona de la página (encabezado, contenido, pie, botón flotante) y, al final, la sección de responsividad. Los nombres de clase se escribieron con guiones (encabezado-principal, vinculos, contenido-principal, contenedor-form, pie-principal, boton-flotante) y los estilos se aplican mediante selectores combinados, por ejemplo .contenido-principal .contenedor-form form label. No se usó JavaScript ni frameworks.

En index-global.css se definió la base visual de todo el sitio. Se aplicó un reseteo con el selector universal (márgenes y rellenos en cero) junto con el modelo de caja alternativo (box-sizing: border-box). Se importó la tipografía Poppins (Regular y Bold) mediante @font-face, con tipografías genéricas como alternativa. Los colores se tomaron del logo; verde oscuro #0F6B66 para títulos, enlaces y pie; verde medio #317F7A para los bordes de las tarjetas; amarillo #F2B134 y naranja #E07A5F para detalles, hover y botones; fondo crema y texto #1F2D2B. El body se organizó como una grilla de tres filas (encabezado, contenido y pie), con el pie al final de la ventana aunque el contenido sea corto. El encabezado combina flexbox con position: sticky y queda fijo al hacer scroll; el menú de navegación es un flex con envoltura de línea y un subrayado amarillo al pasar el mouse. El contenido principal se centró con un ancho máximo de 1100px. En Inicio, las secciones “Destinos destacados” y “Ofertas vigentes” se maquetaron con CSS Grid (repeat(auto-fit, minmax(230px, 1fr))), de modo que las columnas se ajustan solas al ancho disponible. Cada tarjeta lleva fondo blanco, esquinas redondeadas, borde superior de color (verde para destinos, amarillo para ofertas) y una imagen con object-fit: cover. El pie se estilizó con fondo verde y enlaces blancos en fila centrada. Por último, se agregó un enlace flotante con position: fixed en la esquina inferior derecha, con un ícono SVG de mensajería, que lleva a WhatsApp, y se incorporó el favicon en el <head> de las cinco páginas.

Para la versión móvil se armó un menú hamburguesa que se despliega sin JavaScript, solo con HTML y CSS. En el encabezado se agregaron un checkbox oculto y dos etiquetas (label) asociadas a él, boton-abrir y boton-cerrar, que cumplen la función de botones. El estado del menú (abierto o cerrado) queda guardado en el checkbox y se controla con la pseudoclase :checked combinada con el selector de hermano posterior (~). El checkbox se oculta con opacity: 0 y no con display: none, de modo que el menú también se puede abrir desde el teclado, y la pseudoclase :focus-visible marca el botón activo con un contorno amarillo. En pantalla chica, el logo queda a la izquierda y el botón a la derecha, con el

mismo cuadrado de 44px y borde verde. Las tres rayitas del botón de apertura se dibujan con tres degradados de un solo color (linear-gradient) y la cruz del botón de cierre con dos rayitas cruzadas creadas con los pseudoelementos ::before y ::after, por lo que no se usaron imágenes. El menú de navegación permanece oculto (display: none) y, al tildarse el checkbox, se despliega justo debajo del encabezado con position: absolute y top: 100%, con los vínculos uno debajo del otro, fondo blanco, borde inferior amarillo y sombra suave para separarlo del contenido. Al abrirse, el botón de apertura se oculta y el de cierre ocupa su lugar. A partir de los 768px, dentro de la sección de responsividad, los dos botones se ocultan y el menú vuelve a mostrarse en fila al lado del logo.

En catalogo.css se estilizó el panel de filtros como un formulario simple: el form es un flex en columna, las etiquetas van en negrita y verde, y los campos (input, select, textarea) comparten relleno, borde gris claro y esquinas redondeadas. La pseudoclase :focus resalta en verde el campo seleccionado y el botón “Filtrar” cambia a naranja con :hover. La grilla de resultados reutiliza la lógica de tarjetas con auto-fit, e incluye la lista de datos de cada paquete con separadores y el enlace “Ver paquete” resaltado.

En detalle.css se maquetó la galería como un flex con envoltura de línea (datos del paquete a un lado y foto principal al otro). El itinerario se presentó como una lista de tarjetas blancas con borde izquierdo verde. Los bloques “Qué incluye”, “Qué no incluye” y “Carrito” comparten la misma tarjeta blanca y se diferencian por el color del borde superior (verde, naranja y amarillo). El precio final se destacó con mayor tamaño, el botón “Agregar al carrito” quedó verde con hover naranja y el enlace “Continuar” se estilizó como botón amarillo.

En formulario.css el formulario de reserva quedó dentro de una tarjeta blanca centrada de hasta 700px. Cada par etiqueta-campo es un flex en columna y los campos conservan el mismo estilo del catálogo, con :focus verde y botón “Confirmar reserva” con hover naranja. En confirmacion.css se diseñó una tarjeta centrada con el texto alineado al centro, el número de reserva resaltado en naranja oscuro y el enlace “Volver al inicio” convertido en botón.

La responsividad se resolvió con `@media` al final de cada hoja, en la que está el selector que se modifica, con dos puntos de quiebre: 768px y 480px. En `index-global.css`, desde los 768px el menú de navegación se muestra en fila y desaparecen los botones del menú hamburguesa; por debajo de ese ancho el menú queda colapsado y se despliega con el botón. A 768px además se reduce el relleno lateral del encabezado y la altura de la imagen de portada; a 480px se achican los títulos y el logo, el pie apila sus enlaces y el botón flotante se reduce a 48px. En las demás hojas, la galería pasa a columna, las tarjetas de formulario y confirmación reducen su relleno, y los botones y enlaces ocupan todo el ancho en celular.

1.2.5. Implementación con JavaScript

En el archivo index.js se definió la entrada única del código JavaScript, que cargan las 5 páginas con la etiqueta script de tipo module. Allí se importan los demás archivos con import y se obtienen los elementos del DOM por su id con getElementById (encabezado-principal, boton-abrir, boton-cerrar, contenedor-paquetes, form-filtros y form-reserva). Cada bloque se

ejecuta mediante un condicional solo si el elemento existe en la página abierta, por lo que un mismo archivo sirve para todo el sitio sin generar errores. Con el mismo archivo se programó el menú hamburguesa de la versión mobile: el botón de las tres rayas agrega la clase menu-abierto al encabezado mediante addEventListener con el evento click, el botón de la cruz la quita, y la tecla Escape también cierra el menú. En cada cambio se actualiza el atributo aria-expanded del botón de apertura, y los estilos de la clase menu-abierto muestran u ocultan los vínculos.

En paquetes.js se ubicó la estructura de datos del catálogo en un módulo independiente. Es un arreglo de objetos en el que cada paquete tiene los campos id, nombre, destino, imagen, descripción de la imagen, noches, precio, tipo de viaje, fecha de salida, destacado, oferta, itinerario, incluye y no incluye. El arreglo se exporta con export default y se importa en index.js sin llaves.

En catalogo.js se renderizó la sección completa del catálogo desde JavaScript. La función renderizarPaquetes recibe el contenedor y el arreglo de paquetes, recorre el arreglo con forEach, arma cada tarjeta con plantillas de texto (template literals) y las inserta con innerHTML. El precio se muestra con formato de moneda mediante toLocaleString("es-AR") y el enlace "Ver paquete" lleva el id del paquete en la dirección. Si el arreglo está vacío, se muestra un mensaje en lugar de la grilla. La función iniciarCatalogo dibuja todos los paquetes al cargar la página y le pasa a los filtros la función que vuelve a renderizar con el arreglo filtrado. Las dos funciones se exportan de forma nombrada con export.

En filtros.js se programó el filtrado del catálogo. La función filtrarPaquetes recibe el arreglo de paquetes y un objeto de criterios (destino, fecha desde, fecha hasta, precio mínimo, precio máximo y tipos de viaje) y devuelve un arreglo nuevo con el método filter, sin modificar el original. Un criterio vacío no filtra, y los criterios elegidos se combinan. La función iniciarFiltros lee los valores del formulario, escucha el evento submit con addEventListener, evita que la página se recargue con preventDefault y llama a la función de renderizado con el resultado.

En formulario.js se agregaron las validaciones del formulario de reserva. Hay una función para cada campo: el nombre debe tener letras y una longitud mínima, el DNI debe tener 7 u 8 números (se aceptan puntos), el correo debe tener formato válido, el teléfono debe tener entre 8 y 15 números y la fecha de viaje, que es opcional, no puede ser anterior a hoy. Los controles de formato se hacen con expresiones regulares. Los mensajes de error se crean con createElement y se muestran debajo del campo, y se borran cuando el dato se corrige. Se usan los eventos blur para validar al salir de un campo, input para actualizar el error mientras se escribe y submit para validar todo al enviar. Si hay errores, se frena el envío con preventDefault y el foco pasa al primer campo incorrecto. Si todo está correcto, el formulario continúa hacia la página de confirmación.

1.6. Bibliografía

Material teórico:

- Consignas generales

https://drive.google.com/file/d/1M7IbinLC8mvjJ7zEPrqqwjf3g0IpekCl/view?usp=sharing

- Guía teórica: https://drive.google.com/file/d/1Bel3TTivym9l8D7zJ5hVJnZL87Rs9qQ4/view?usp=sharing

Recursos técnicos:

- NIC Argentina: https://nic.ar/
- DonWeb: https://donweb.com/es-ar/
- MDN Web Docs: https://developer.mozilla.org/en-US/
- Coyier, C. A Complete Guide to CSS Flexbox. CSS-Tricks: https://css-tricks.com/snippets/css/a-guide-to-flexbox/
- MDN JavaScript: JavaScript | MDN

Herramientas utilizadas:

- Gemini: https://gemini.google.com/
- Claude: https://claude.ai/
- Canva: https://www.canva.com/
- CamScanner: https://www.camscanner.com/es-es
- Figma: https://www.figma.com
- GoogleFonts: https://fonts.google.com/